

UNIT II Requirements Engineering and Analysis

Syllabus

Software requirements, types of requirements (functional and non-functional), requirement elicitation techniques (interviews, questionnaires, observation, brainstorming, workshops), feasibility study (technical, operational, economic), requirement analysis, Software Requirement Specification (SRS) document, components of analysis model, user stories, requirement validation and verification, Case Study: Hospital Management System.

Software requirements

Classification of Software Requirements

Classification of Software Requirements is important in the software development process. It organizes our requirements into different categories that make them easier to manage, prioritize, and track. The main types of Software Requirements are functional, non-functional, and domain requirements.

What is Software Requirements?

According to IEEE standard 729, a requirement is defined as follows:

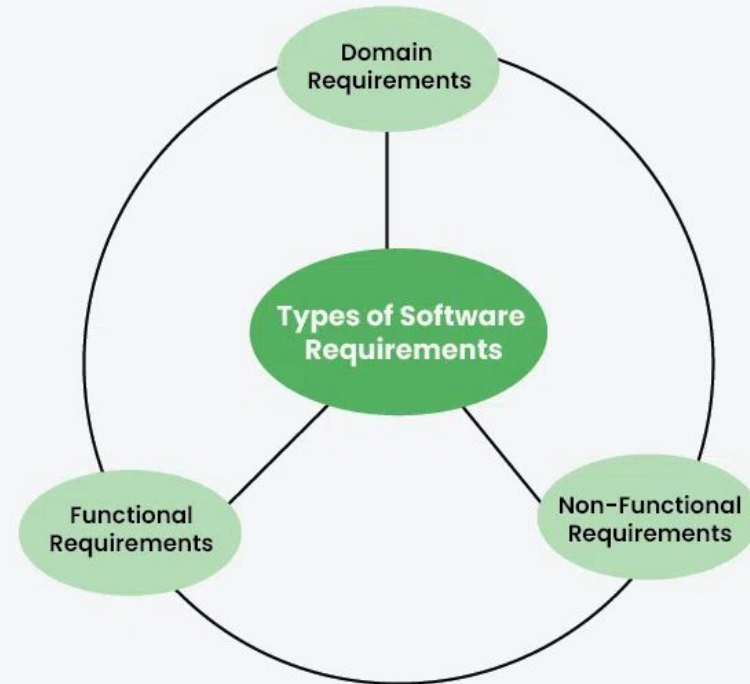
- A condition or capability needed by a user to solve a problem or achieve an objective
- A condition or capability that must be met or possessed by a system or system component to satisfy a contract, standard, specification or other formally imposed documents
- A documented representation of a condition or capability, as in 1 and 2.

Types of Software Requirements

Software Requirements are mainly classified into three types:

- Functional requirements
- Non-functional requirements
- Domain requirements

Types of software Requirements



1. Functional Requirements

Definition: Functional requirements describe what the software should do. They define the functions or features that the system must have.

Examples:

- **User Authentication:** The system must allow users to log in using a username and password.
- **Search Functionality:** The software should enable users to search for products by name or category.
- **Report Generation:** The system should be able to generate sales reports for a specified date range.

Explanation: Functional requirements specify the actions that the software needs to perform. These are the basic features and functionalities that users expect from the software.

2. Non-functional Requirements

Definition: Non-functional requirements describe how the software performs a task rather than what it should do. They define the quality attributes, performance criteria, and constraints.

Examples:

- **Performance:** The system should process 1,000 transactions per second.
- **Usability:** The software should be easy to use and have a user-friendly interface.
- **Reliability:** The system must have 99.9% uptime.
- **Security:** Data must be encrypted during transmission and storage.

Explanation: Non-functional requirements are about the system's behavior, quality, and constraints. They ensure that the software meets certain standards of performance, usability, reliability, and security.

3. Domain Requirements

Definition: Domain requirements are specific to the domain or industry in which the software operates. They include terminology, rules, and standards relevant to that particular domain.

Examples:

- **Healthcare:** The software must comply with HIPAA regulations for handling patient data.
- **Finance:** The system should adhere to GAAP standards for financial reporting.
- **E-commerce:** The software should support various payment gateways like PayPal, Stripe, and credit cards.

Explanation: Domain requirements reflect the unique needs and constraints of a particular industry. They ensure that the software is relevant and compliant with industry-specific regulations and standards.

What are Functional Requirements?

Functional Requirements are the requirements that the end user specifically demands as basic facilities that the system should offer. It can be a calculation, data manipulation, business process, user interaction, or any other specific functionality that defines what function a system is likely to perform. All these functionalities need to be necessarily incorporated into the system as a part of the contract. These are represented or stated in the form of input to be given to the system, the operation performed and the output expected.

- They are the requirements stated by the user which one can see directly in the final product, unlike the non-functional requirements. For example, in a hospital management system, a doctor should be able to retrieve the information of his patients.
- Each high-level functional requirement may involve several interactions or dialogues between the system and the outside world.
- To accurately describe the functional requirements, all scenarios must be enumerated.
- There are many ways of expressing functional requirements e.g., natural language, a structured or formatted language with no rigorous syntax, and formal specification language with proper syntax.
- Functional Requirements in Software Engineering are also called Functional Specification.

What are Non-functional Requirements?

These are basically the quality constraints that the system must satisfy according to the project contract. Nonfunctional requirements, not related to the system functionality, rather define how the system should perform. The priority or extent to which these factors are implemented varies from one project to other. They are also called non-behavioral requirements. They basically deal with issues like:

- **Portability**
- **Security**
- **Maintainability**
- **Reliability**
- **Scalability**
- **Performance**
- **Reusability**
- **Flexibility**

Non-functional requirements are classified into the following types:

- Interface constraints
- Performance constraints: response time, security, storage space, etc.
- Operating constraints
- Life cycle constraints: maintainability, portability, etc.
- Economic constraints

The process of specifying non-functional requirements requires the knowledge of the functionality of the system, as well as the knowledge of the context within which the system will operate.

They are divided into two main categories

- **Execution qualities:** These consist of things like security and usability, which are observable at run time.
- **Evolution qualities:** These consist of things like testability, maintainability, extensibility, and scalability that are embodied in the static structure of the software system.

What are Domain requirements?

- ❑ Domain requirements are the requirements that are characteristic of a particular category or domain of projects.
- ❑ Domain requirements can be functional or nonfunctional. Domain requirements engineering is a continuous process of proactively defining the requirements for all foreseeable applications to be developed in the software product line.
- ❑ The basic functions that a system of a specific domain must necessarily exhibit come under this category.
- ❑ For instance, in academic software that maintains records of a school or college, the functionality of being able to access the list of faculty and list of students of each grade is a domain requirement. These requirements are therefore identified from that domain model and are not user-specific.

Classifications of Software Requirements

Other common classifications of software requirements can be:

- 1.User requirements:** These requirements describe what the end-user wants from the software system. User requirements are usually expressed in natural language and are typically gathered through interviews, surveys, or user feedback.
- 2.System requirements:** These requirements specify the technical characteristics of the software system, such as its architecture, hardware requirements, software components, and interfaces. System requirements are typically expressed in technical terms and are often used as a basis for system design.
- 3.Business requirements:** These requirements describe the business goals and objectives that the software system is expected to achieve. Business requirements are usually expressed in terms of revenue, market share, customer satisfaction, or other business metrics.
- 4.Regulatory requirements:** These requirements specify the legal or regulatory standards that the software system must meet. Regulatory requirements may include data privacy, security, accessibility, or other legal compliance requirements.

5.Interface requirements: These requirements specify the interactions between the software system and external systems or components, such as databases, web services, or other software applications.

6.Design requirements: These requirements describe the technical design of the software system. They include information about the software architecture, data structures, algorithms, and other technical aspects of the software.

By classifying software requirements, it becomes easier to manage, prioritize, and document them effectively. It also helps ensure that all important aspects of the system are considered during the development process.

Advantages of Classifying Software Requirements

Advantages of classifying software requirements include:

- 1. Better organization:** Classifying software requirements helps organize them into groups that are easier to manage, prioritize, and track throughout the development process.
- 2. Improved communication:** Clear classification of requirements makes it easier to communicate them to stakeholders, developers, and other team members. It also ensures that everyone is on the same page about what is required.
- 3. Increased quality:** By classifying requirements, potential conflicts or gaps can be identified early in the development process. This reduces the risk of errors, omissions, or misunderstandings, leading to higher-quality software.
- 4. Improved traceability:** Classifying requirements helps establish traceability, which is essential for demonstrating compliance with regulatory or quality standards.

Disadvantages of classifying software requirements

Disadvantages of classifying software requirements include:

- 1.Complexity:** Classifying software requirements can be complex, especially if there are many stakeholders with different needs or requirements. It can also be time-consuming to identify and classify all the requirements.
- 2.Rigid structure:** A rigid classification structure may limit the ability to accommodate changes or evolving needs during the development process. It can also lead to a siloed (Being separated or isolated into distinct , self-contained) approach that prevents the integration of new ideas or insights.
- 3.Misclassification:** Misclassifying requirements can lead to errors or misunderstandings that can be costly to correct later in the development process.

Requirements Elicitation - Software Engineering

Requirements elicitation is the process of gathering and defining the requirements for a software system. The goal of requirements elicitation is to ensure that the software development process is based on a clear and comprehensive understanding of the customer's needs and requirements.

What is Requirement Elicitation?

The process of investigating and learning about a system's requirements from users, clients, and other stakeholders is known as requirements elicitation. Requirements elicitation in software engineering is perhaps the most difficult, most error-prone, and most communication-intensive software development.

1. Requirement Elicitation can be successful only through an effective customer-developer partnership. It is needed to know what the users require.
2. Requirements elicitation involves **the identification, collection, analysis, and refinement of the requirements** for a software system.
3. Requirement Elicitation is a critical part of the software development life cycle and is typically performed at the beginning of the project.

- 4.Requirements elicitation involves stakeholders from different areas of the organization, including business owners, end-users, and technical experts.
- 5.The output of the requirements elicitation process is a set of clear, concise, and well-defined requirements that serve as the basis for the design and development of the software system.
- 6.Requirements elicitation is difficult because just questioning users and customers about system needs may not collect all relevant requirements, particularly for safety and dependability.
- 7.Interviews, surveys, user observation, workshops, brainstorming, use cases, role-playing, and prototyping are all methods for eliciting requirements.**

Importance of Requirements Elicitation

- 1.Compliance with Business Objectives:** The process of elicitation guarantees that the software development endeavors are in harmony with the wider company **aims and objectives**. Comprehending the business context facilitates the development of a solution that adds value for the company.
- 2.User Satisfaction:** It is easier to create software that fulfills end users' needs and expectations when they are involved in the requirements elicitation process. Higher user pleasure and acceptance of the finished product are the results of this.
- 3.Time and Money Savings:** Having precise and well-defined specifications aids in preventing miscommunication and rework during the development phase. As a result, there will be cost savings and the project will be completed on time.
- 4.Compliance and Regulation Requirements:** Requirements elicitation is crucial for projects in regulated industries to guarantee that **the software conforms with applicable laws and norms. In industries like healthcare, finance, and aerospace**, this is crucial.
- 5.Traceability and Documentation:** Throughout the software development process, traceability is based on well-documented requirements. **Traceability helps with testing, validation, and maintenance by ensuring that every part of the software can be linked to a particular requirement.**

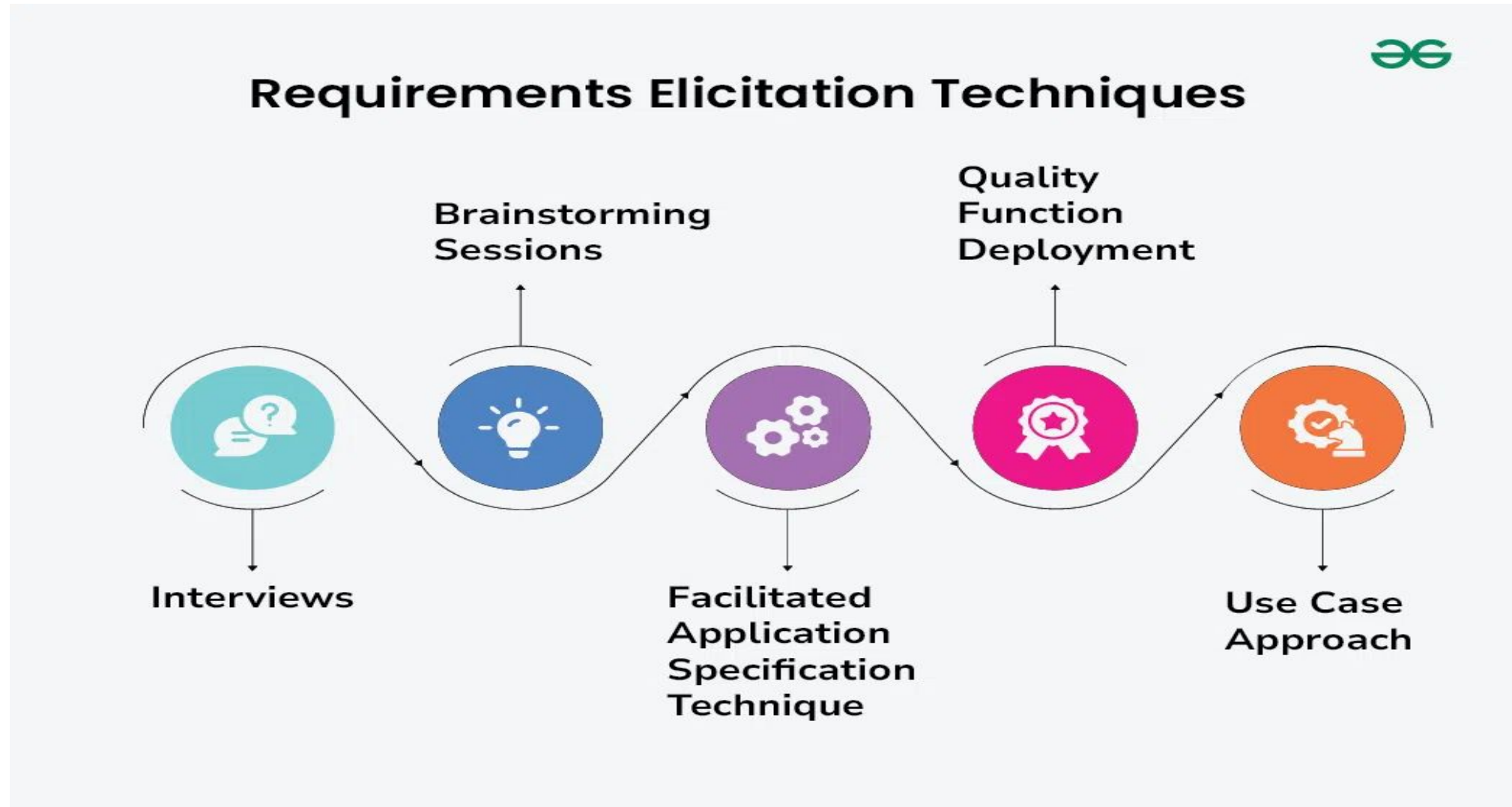
Requirements Elicitation Activities

Requirements elicitation includes the subsequent activities. A few of them are listed below:

1. **Knowledge of the overall area** where the systems are applied.
2. The details of the **precise customer problem where the system is going to be applied must be understood.**
3. Interaction of system with **external requirements.**
4. Detailed **investigation of user needs.**
5. Define the **constraints for system development.**

Requirements Elicitation Methods

There are several requirements elicitation methods. A few of them are listed below:



1. Interviews

The objective of conducting an interview is to understand the customer's expectations of the software. It is impossible to interview every stakeholder hence representatives from groups are selected based on their expertise and credibility. Interviews may be open-ended or structured.

1. In open-ended interviews, there is no pre-set agenda. Context-free questions may be asked to understand the problem.
2. In a structured interview, an agenda of fairly open questions is prepared. Sometimes a proper questionnaire is designed for the interview.

2. Brainstorming Sessions

- Brainstorming Sessions is a group technique
- It is intended to generate lots of new ideas hence providing a platform to share views
- A highly trained facilitator is required to handle group bias and conflicts.
- Every idea is documented so that everyone can see it.
- Finally, a document is prepared which consists of the list of requirements and their priority if possible.

3. Facilitated Application Specification Technique

Its objective is to **bridge the expectation gap** - the difference between **what the developers think they are supposed to build** and **what customers think they are going to get**. A team-oriented approach is developed for requirements gathering. Each attendee is asked to make a list of objects that are:

1. Part of the environment that surrounds the system.
2. Produced by the system.
3. Used by the system.

Each participant prepares his/her list, different lists are then combined, redundant entries are eliminated, the team is divided into smaller sub-teams to develop mini-specifications and finally, a draft of specifications is written down using all the inputs from the meeting.

4. Quality Function Deployment

In this technique customer satisfaction is of prime concern, hence it emphasizes the requirements that are valuable to the customer.

3 types of requirements are identified:

- **Normal requirements:** In this the objective and goals of the proposed software are discussed with the customer. For example - normal requirements for a result management system may be entry of marks, calculation of results, etc.
- **Expected requirements:** These requirements are so obvious that the customer need not explicitly state them. Example - protection from unauthorized access.
- **Exciting requirements:** It includes features that are beyond customer's expectations and prove to be very satisfying when present. For example - when unauthorized access is detected, it should back up and shut down all processes.

5. Use Case Approach

Use Case technique combines text and pictures to provide a better understanding of the requirements.

The use cases describe the 'what', of a system and not 'how'. Hence, they only give a functional view of the system.

The components of the use case design include three major things - Actor, use cases, and use case diagram.

1.Actor: It is the external agent that lies outside the system but interacts with it in some way. An actor may be a person, machine, etc. It is represented as a stick figure. Actors can be primary actors or secondary actors.

1. Primary actors: It requires assistance from the system to achieve a goal.

2. Secondary actor: It is an actor from which the system needs assistance.

2.Use cases: They describe the sequence of interactions between actors and the system. They capture who(actors) do what(interaction) with the system. A complete set of use cases specifies all possible ways to use the system.

3. Use case diagram: A use case diagram graphically represents what happens when an actor interacts with a system. It captures the functional aspect of the system.

3. A stick figure is used to represent an actor.
4. An oval is used to represent a use case.
5. A line is used to represent a relationship between an actor and a use case.

The success of an elicitation technique used depends on the maturity of the analyst, developers, users, and the customer involved.

Steps of Requirements Elicitation



Steps of Requirements Elicitation



1. Identify Stakeholders

The first step is to figure out who all the key players are in the project. These are the people who will either use the product, help develop it, or be impacted by it. This could include users, developers, project managers, and customers. It's important to get everyone involved early on, so you gather all the necessary input and perspectives when defining the requirements.

2. Gather Requirements

Once you've identified the stakeholders, the next step is to gather their requirements. This means understanding what they need from the system or product. Requirements can be functional (what the system should do) or non-functional (how it should perform). You can use different methods like interviews, surveys, or group discussions to collect all these needs.

3. Prioritize Requirements

Not all requirements are equal in terms of importance. After gathering them, you'll need to prioritize which ones should be addressed first. This helps ensure that the most important features are built into the system first, especially when resources (like time and budget) are limited. A common approach is to categorize them into things that are "Must-have," "Should-have," "Could-have," and "Won't-have."

4. Categorize Feasibility

After prioritizing, the next step is to assess how realistic each requirement is. You'll categorize them into three groups:

- **Achievable:** These are requirements that are realistic and can be done within the available resources (time, budget, etc.).
- **Deferred:** These requirements are important but can't be implemented in this phase. They'll be put on hold for now with a clear reason why.
- **Impossible:** These requirements can't be done because of technical or other limitations, and they should be removed from the list.

Software Requirement Specification (SRS) Format

In order to form a good SRS, here you will see some points that can be used and should be considered to form a structure of good Software Requirements Specification (SRS). These are below mentioned in the table of contents and are well explained below.

Software Requirement Specification (SRS) Format as the name suggests, is a complete specification and description of requirements of the software that need to be fulfilled for the successful development of the software system. These requirements can be functional as well as non-functional depending upon the type of requirement. The interaction between different customers and contractors is done because it is necessary to fully understand the needs of customers.

Document Title

Author(s)

Affiliation

Address

Date

Document Version

Depending upon information gathered after interaction, SRS is developed which describes requirements of software that may include changes and modifications that is needed to be done to increase quality of product and to satisfy customer's demand.

Introduction

- **Purpose of this Document** - At first, main aim of why this document is necessary and what's purpose of document is explained and described.
- **Scope of this document** - In this, overall working and main objective of document and what value it will provide to customer is described and explained. It also includes a description of development cost and time required.
- **Overview** - In this, description of product is explained. It's simply summary or overall review of product.

General description

In this, general functions of product which includes objective of user, a user characteristic, features, benefits, about why its importance is mentioned. It also describes features of user community.

Functional Requirements

In this, possible outcome of software system which includes effects due to operation of program is fully explained. All functional requirements which may include calculations, data processing, etc. are placed in a ranked order. Functional requirements specify the expected behavior of the system-which outputs should be produced from the given inputs. They describe the relationship between the input and output of the system. For each functional requirement, detailed description all the data inputs and their source, the units of measure, and the range of valid inputs must be specified.

Interface Requirements

In this, software interfaces which mean how software program communicates with each other or users either in form of any language, code, or message are fully described and explained. Examples can be shared memory, data streams, etc.

Performance Requirements

In this, how a software system performs desired functions under specific condition is explained. It also explains required time, required memory, maximum error rate, etc. The performance requirements part of an SRS specifies the performance constraints on the software system. All the requirements relating to the performance characteristics of the system must be clearly specified. There are two types of performance requirements: static and dynamic. Static requirements are those that do not impose constraint on the execution characteristics of the system. Dynamic requirements specify constraints on the execution behaviour of the system.

Design Constraints

In this, constraints which simply means limitation or restriction are specified and explained for design team. Examples may include use of a particular algorithm, hardware and software limitations, etc.

There are a number of factors in the client's environment that may restrict the choices of a designer leading to design constraints such factors include standards that must be followed resource limits, operating environment, reliability and security requirements and policies that may have an impact on the design of the system. An SRS should identify and specify all such constraints.

Non-Functional Attributes

In this, non-functional attributes are explained that are required by software system for better performance. An example may include Security, Portability, Reliability, Reusability, Application compatibility, Data integrity, Scalability capacity, etc.

Preliminary Schedule and Budget

In this, initial version and budget of project plan are explained which include overall time duration required and overall cost required for development of project.

Appendices

In this, additional information like references from where information is gathered, definitions of some specific terms, acronyms, abbreviations, etc. are given and explained.

Uses of SRS document

- Development team require it for developing product according to the need.
- Test plans are generated by testing group based on the describe external behaviour.
- Maintenance and support staff need it to understand what the software product is supposed to do.
- Project manager base their plans and estimates of schedule, effort and resources on it.
- customer rely on it to know that product they can expect.
- As a contract between developer and customer.
- in documentation purpose.

Verification and Validation in Software

Engineering

Verification and Validation are the processes of investigating whether a software system satisfies specifications and standards and fulfills the required purpose. Verification and Validation both play an important role in developing good software. **Verification helps in examining whether the product is built right according to requirements, while validation helps in examining whether the right product is built to meet user needs.**

Here we are learning about the Verification and Validation in detail:

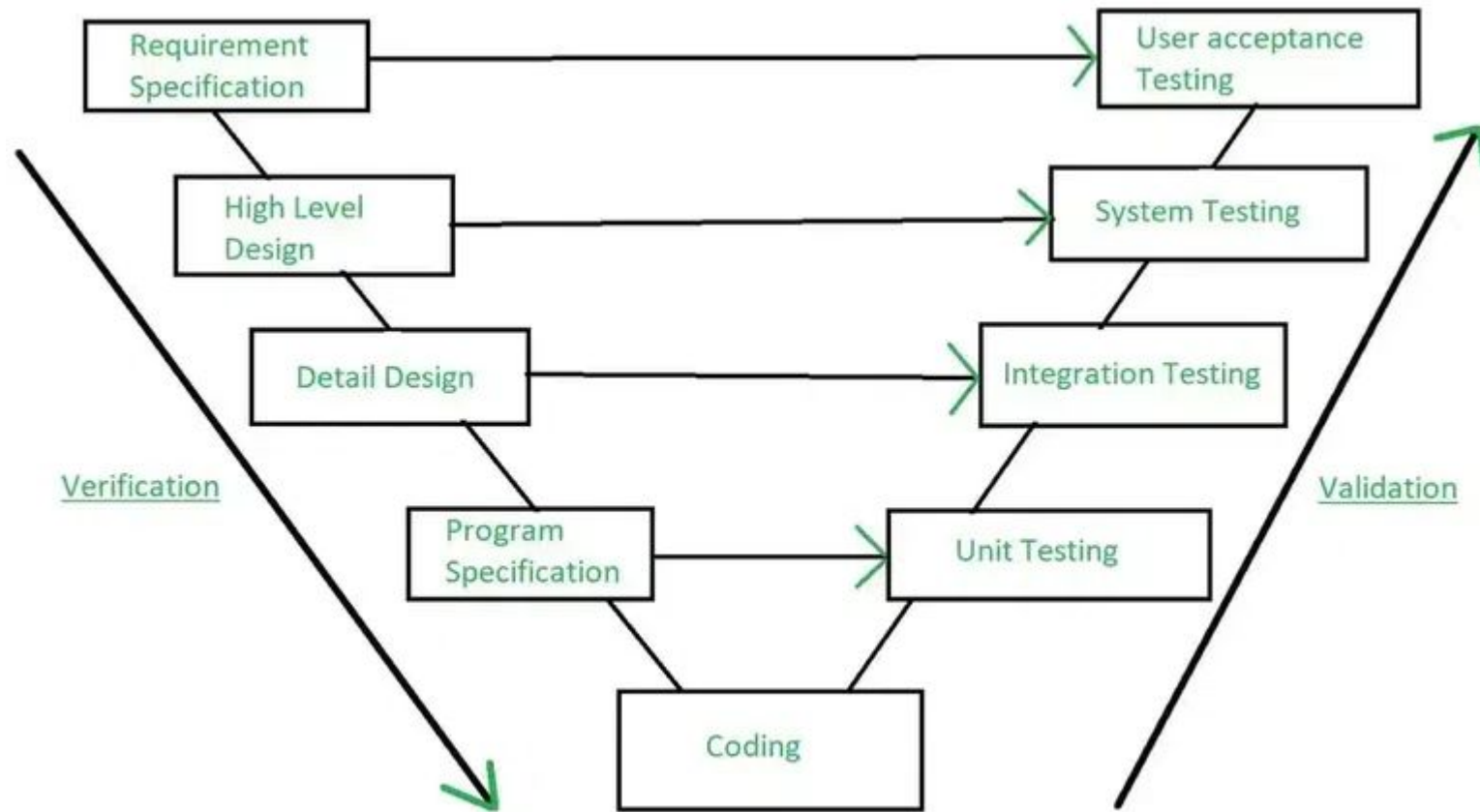
What is Verification?

Verification is the process of checking that software achieves its goal without any bugs. It is the process to ensure whether the product that is developed is right or not. It verifies whether the developed product fulfills the requirements that we have. Verification is static testing. Verification means **Are we building the product right?**

Static Testing

Verification Testing is known as Static Testing, and it can be simply termed as checking whether we are developing the right product or not, and also whether our software is fulfilling the customer's requirement or not. Here are some of the activities that are involved in verification.

- **Inspections:** This is when we carefully look over the software's design or code to spot any potential problems or areas that could be improved.
- **Reviews:** This is a team effort, where everyone gets together to assess the software's requirements, design, or code. The goal is to make sure everything is on track and meets the specified objectives.
- **Walkthroughs:** In a walkthrough, the developer or designer gives a casual presentation of their work, explaining how it meets the project's requirements. It's a great way to get everyone on the same page.
- **Desk-Checking:** This is when a developer manually reviews their own code or design to catch any mistakes or issues before moving forward.



Verification and Validation

What is Validation?

Validation is the process of checking whether the **Software Product** is up to the mark, or in other words product has high-level requirements. It is the process of checking the validation of the product i.e., it checks whether what we are developing is the right product. It is a validation of the actual and expected products. Validation is dynamic testing. Validation means **Are we building the right product?**

Dynamic Testing

Validation Testing is known as **Dynamic Testing** in which we examine whether we have developed the product right or not and also about the business needs of the client. Here are some of the activities that are involved in Validation.

- **Black Box Testing:** This is all about testing the software from the user's perspective. You don't need to know how the code works internally. Instead, you just focus on whether the software behaves as expected when you use it, like clicking buttons or entering information.
- **White Box Testing:** In this case, you have access to the internal workings of the software. You're testing the logic, the flow, and how the code is structured to make sure everything is functioning as it should behind the scenes.

- Unit Testing:** This involves testing individual parts or functions of the software to make sure each piece works properly on its own, before they're combined with other parts of the system.
- Integration Testing:** Once individual pieces are tested, integration testing checks how well different parts of the software work together. It's like making sure all the components play nicely when combined.

Example Scenario of Verification and Validation

Verification: During development, the team takes a step back to check the system's design and code. They focus on key features like user registration, the shopping cart, and payment processing. Through inspections and walkthroughs, they make sure each part of the software is being built according to the initial plan and specifications.

Validation: Once the software is finished, it's time to test it. First, the team runs **unit tests** to check individual features, like making sure the shopping cart calculates prices correctly. Then, **integration tests** ensure all features work well together. After that, **system testing** checks if everything functions as a whole. Finally, the client does **acceptance testing** to confirm that the software meets their needs and expectations.

Differences between Verification and Validation

	Verification	Validation
Definition	Verification refers to the set of activities that ensure software correctly implements the specific function	Validation refers to the set of activities that ensure that the software that has been built is traceable to customer requirements.
Focus	It includes checking documents, designs, codes, and programs.	It includes testing and validating the actual product.
Type of Testing	Verification is the Static testing.	Validation is Dynamic testing.
Execution	It does <i>not</i> include the execution of the code.	It includes the execution of the code.
Methods Used	Methods used in verification are reviews, walkthroughs, inspections and desk-checking.	Methods used in validation are Black Box Testing, White Box Testing and Non-Functional testing.